data, on the left with linear scale and on the right using logarithmic scale for heights.

There are two reasons for this difference:

- Having a high duplicates rate (larger and larger as `n` is growing) produces sequence of insertions with longer streaks of sorted data;
- BSTs are naturally left-leaning when duplicates are allowed: as you can also see in figure listing 3.5, we need to break ties when inserting new keys, and we decided to go left every time we find a duplicate. Unfortunately in this case we need a deterministic decision, we can't use the same workaround as for deletions, and so when input sequences contains a large number of duplicates, like in this test, BSTs becomes tangibly skewed.

This is already a nice result for treaps, as real-world inputs can easily contain several duplicates, in many applications.

But, what about those applications that doesn't allow duplicates? Are we safe from adversarial sequences in those situations? Truth to be told, we haven't clarified how important is the role of ordering: then, what better way to do it than trying directly the worst possible case, a totally-ordered sequence? Figure 3.19 shows the results for this test: here we stripped out all randomness, and just added to the containers all integers from `0` to `n-1`, no calls to remove performed.